Facility Usage Prediction System: Technical Documentation

All five sections are complete. Reproduce everything with the commands in `README.md`.

1. Data

1.1 Why synthetic, and how it is built

The brief asks for realistic booking history for one residential community. No real data is available, so `src/facility_pred/data_gen.py` simulates one calendar year (2025-01-01 to 2025-12-31) for 300 residents and 6 facilities. It is seeded (`--seed 42`), so the same command always produces the same file, and CI checks that the regenerated `data/bookings.csv` is byte-identical to the committed one.

Schema (`data/bookings.csv`, one row per booking): `booking_id`, `resident_id`, `facility_id`, `booking_ts` (when the booking was created), `usage_ts` (when the facility is used). `data/facilities.csv` maps `facility_id` to name and operating hours.

1.2 Properties the brief asked for, and how each is produced

Property	How it is simulated
Resident preferences	Each resident draws a sparse facility preference (Gamma-weighted popularity), 2-3 favourite weekdays, and a peak hour per facility.
Facility popularity / imbalance	Base shares: Gym 34%, Pool 22%, Badminton 16%, Tennis 12%, Clubhouse 9%, Hall 7%.
Time patterns	Peak hours differ by facility (gym 6-7am and 6-7pm; pool 7am and 5-6pm; etc.). Pool demand is x1.8 from March to September. Clubhouse and Hall demand is x2 at weekends.
Booking lead times	A resident-specific routine books at a habitual time of day, 0-3 days ahead (50-90% of their bookings). The rest are ad hoc with a log-normal lead time (median 14h, capped 0.5-240h), always made while awake (07:00-22:59).
Sparsity	Booking rate per resident is log-normal (heavy-tailed). About 30% of residents move in during the year and 12% leave.
Noise	10% of bookings go to a random facility. Hours spread +/-1h around the peak, with 5% fully random. Booking time has a ~8 minute jitter.
Changing behaviour	28% of residents switch to new facility preferences and shifted hours at a random changepoint.

1.3 How it was verified

`python -m facility_pred.data_report` prints the distributions below, and a second pass checked the data directly. Results on the committed file (15,122 bookings, 300 residents):

- Integrity: no nulls, no duplicate booking IDs, no resident booked twice at the same time, every `booking_ts` is before its `usage_ts`, every usage hour is within the facility's opening hours, and no booking is made between 23:00 and 06:59.
- Imbalance: Gym 24%, Pool 22%, Badminton 16%, Clubhouse 14%, Tennis 13%, Hall 11% of bookings.
- Sparsity: median 38 bookings per resident (min 1, max 171); 12% of residents have fewer than 10.

- Preferences: among residents with 10+ bookings, the favourite facility averages 49% of their bookings.
- Drift: 35% of residents with 30+ bookings have a different favourite facility in the second half of their history than in the first.
- Predictability: repeating the previous facility is right 37% of the time and the previous hour 26%, so simple baselines do not solve the task. Booking clock time is within +/-30 minutes of a resident's median in 64% of bookings, which caps how well the notification time can be predicted.
- Seasonality: pool demand is higher from March to September; monthly volume is roughly flat (about 1,050 in January to about 1,450 in December).
- Two fixes made during verification: (1) a first version had all residents joining at random dates, so January had about 140 bookings against about 1,300 by August; most residents (70%) are now present from the start. (2) On reviewing the first review spreadsheet, about 10% of bookings had been made between midnight and 6am (ad hoc lead times ignored the time of day). Ad hoc bookings are now made only between 07:00 and 22:59. The dataset was regenerated and the pipeline rerun after fix (2); see section 5 for what that means for the test split.

1.4 Limitations

- Results on this data show the pipeline works, not how it performs on real resident behaviour; the habits the models learn are the habits the generator planted.
- No facility capacity limits, cancellations, holidays, or multi-year seasonality.
- Resident habits are independent of each other; there are no household or group bookings.

2. Problem framing, features and leakage controls

2.1 Framing

For every booking, predict the resident's **next booking** from their history up to and including their previous booking. The prediction time T is therefore the previous booking's `booking_ts`. Four outputs:

Output	Target	Model	Counted as a hit when
Facility	one of 6	gradient-boosted classifier	exact
Usage day	weekday (Mon-Sun)	gradient-boosted classifier	exact
Usage hour	hour of day	gradient-boosted classifier	exact (+/-1 h also reported)
Notification ("nudge")	weekday + clock time at which the resident books	derived (below)	weekday exact **and** clock within 30 min

The nudge follows the brief's example, where it equals the time the resident actually booked. It is derived from two predictions: *nudge day = (predicted usage day - predicted lead days) mod 7* (lead days is a fourth classifier: calendar days between booking and use, capped at 7) and *nudge time = a rule* (see 3.2). A row's **score** is the number of the four outputs that hit (k of 4); a row **matches** only when all four hit. The overall measure is the mean of k/4 across rows.

2.2 Features (98, all computed from the resident's own earlier bookings)

- **Recency-weighted habit histograms** (decay 0.9 per booking): share of each facility, usage weekday, usage hour, lead days, and the weekday on which the resident tends to *book*. A cumulative (non-decayed) facility share is also kept.
- **Per-facility habits**: modal hour and modal weekday for each facility, and for the resident's top-2 facilities.
- **Last booking**: facility, weekday, hour, lead time, booking clock time and weekday; the previous two facilities.

- **Rhythm**: number of prior bookings, days since the first booking, median and last gap between bookings, booking month.
- **Booking time**: median of the last 10 booking times, their spread, and the densest 30-minute cluster of booking times (`clock_mode`).

2.3 Leakage controls

1. **Chronological split by the target booking's `booking_ts`**: train before 2025-09-15, validation 2025-09-15 to 2025-10-31 (used only to choose hyper-parameters and design decisions), **test from 2025-11-01** (2,736 bookings). After tuning, the models are refit on train + validation; the test split is never used for fitting or tuning.
2. **Features are emitted before the row is folded into history.** `features.build` makes one forward pass over each resident's bookings in time order; a row's features are written first and only then added to the resident's running state. Features cannot depend on the row itself or later rows.
3. **Automated check** (`tests/test_features.py`): scrambling the facility and usage time of every booking from row k onward leaves the features of rows 0..k unchanged. A second test asserts that each resident's first booking has no history, a third that the split is chronological and disjoint. CI runs these.
4. Habit state is per resident only: no statistic is computed across the whole dataset, so no test-period information enters the training features.
5. Cold start: a resident's first booking has no features beyond a count of zero. The model must rely on global patterns for those rows (none occur in the test split, since all test residents had earlier bookings).

3. Modelling

3.1 Approach

Scikit-learn `HistGradientBoostingClassifier` for facility, usage day, usage hour and lead days. It handles missing values natively (residents with no use of a facility), trains in seconds on ~12k rows, and needs no feature scaling. Hyper-parameters (learning rate, leaves, L2; 8 combinations) were chosen per target on the validation split. Chosen values are saved in `models/chosen_params.json`. Everything is seeded; retraining reproduces `results/metrics.json` byte for byte.

3.2 Booking clock time: a rule, not a model

The first version used a boosted regressor with absolute-error loss. On validation it reached 53.8% within 30 minutes, **worse** than the "median of last 10 booking times" baseline (56.3%). Booking times are bimodal (habitual time plus ad hoc bookings), and a median sits between the two. The final rule takes the densest 30-minute cluster of the resident's own recency-weighted booking times (63.2% on validation); it optimises the metric directly and is easy to explain.

3.3 Baselines (computed from the same history, no learning)

`majority` (training-set mode), `repeat_last` (copy the previous booking), and `resident_habit` (the resident's own recency-weighted modal facility, hour, weekday, lead time and clock cluster).

3.4 Alternatives considered

- **Direct nudge-day classifier**: tried on validation; 22.6% vs 24.0% for the derived day, so dropped for simplicity.
- **Sequence models (RNN / transformer)** and **Markov chains over facilities**: not tried. With ~12k rows and 300 residents, hand-built habit features are the stronger bet; a sequence model would need far more data per resident.

- **Collaborative filtering across residents**: not tried; residents' habits are independent in this data, so there is nothing for neighbours to teach.
- **LightGBM / XGBoost**: similar model family; scikit-learn's histogram booster avoids an extra native dependency.

4. Evaluation on the unseen test split

Test split: 2,736 bookings made on or after 2025-11-01, scored with the final models (refit on train + validation).

	Facility	Usage day	Usage hour	Nudge	Mean k of 4	All 4 match
Gradient boosting	45.1%	**42.5%**	39.6%	**17.4%**	**36.1%**	**3.7%**
Baseline: resident habit	**47.6%**	27.5%	39.8%	12.7%	31.9%	2.9%
Baseline: repeat last	37.6%	12.5%	26.7%	6.3%	20.8%	0.5%
Baseline: majority	26.4%	15.1%	17.0%	0.7%	14.8%	0.0%

Supporting metrics for the model:

Output	Metrics
Facility	accuracy 45.1%, macro-F1 41.6%, top-3 accuracy 82.0%
Usage day	accuracy 42.5%, within one day 63.1%
Usage hour	accuracy 39.6%, within one hour 58.4%, MAE 3.49 h
Nudge	hit 17.4%: nudge weekday 27.5%, booking time within 30 min 62.9% (within 60 min 66.2%, MAE 121 min); a hit needs both

The per-record comparison is in `outputs/prediction_review.xlsx` (sheet *Predictions*, one row per test booking, with the colour-coded match flags and "k of 4" score; sheet *Summary* holds every table in this section as numeric cells). A CSV copy is in `outputs/prediction_review.csv`.

4.1 Honest reading of the results

- The model has the best overall score (36.1% vs 31.9% for the strongest baseline) and clear wins on usage day (42.5% vs 27.5%) and nudge (17.4% vs 12.7%).
- **It does not beat the resident-habit baseline on facility (45.1% vs 47.6%) and only ties on hour (39.6% vs 39.8%).** Those two outputs are close to what the planted habits allow: a resident's favourite facility is only about half of their bookings, 10% of bookings go to a random facility, and 28% of residents change habits mid-year.
- Only 3.7% of test bookings match on all four outputs. The nudge is the limiting output: the booking time is right 63% of the time, but the booking *weekday* is right only 27% of the time, and both are required.
- The headline numbers depend on the strict hit definitions: top-3 facility accuracy is 82.0% and hour within one hour is 58.4%.

4.2 Error analysis

- **Class imbalance shows up as a Gym bias.** Gym is predicted for 938 of 2,736 test bookings but occurs in 723. Gym recall is 63%, Pool 52%, Badminton 44%, Clubhouse 37%, Tennis 29% and Multipurpose Hall 23%.
- **More history helps.** Mean k/4 is 36.7% for residents with 30+ prior bookings, 33.6% for 10-29 and 31.2% for 1-9 (only 56 test rows, so that last figure is noisy).
- **Hour errors follow facility errors**: hour accuracy is 55% for Gym bookings and 18% for the Hall (31% Clubhouse), because each facility has its own peak hours.

- **Nudge errors are mostly the weekday.** The model has no information on which day a resident will decide to book; it can only infer it from the usage day and lead time, and both are themselves uncertain.

5. Limitations

- All results are on simulated data; they show the pipeline works, not the accuracy to expect on real residents.
- The data has no capacity limits, cancellations, holidays or multi-year seasonality (see 1.4).
- A single chronological holdout (about 2 months) gives one estimate with no confidence interval; a rolling-origin evaluation would be more robust.
- The test split was scored several times, under two versions of the dataset. After the first scoring I reviewed the review spreadsheet, found overnight bookings, fixed the generator and reran everything; the earlier version scored 34.9% mean k/4 against 36.1% now, with the same conclusions. The fix was a realism correction, not a tuning step, but the test split is not pristine. No model or feature decision used test numbers; design decisions (the clock rule, dropping the direct nudge day) used the validation split only.
- Prediction is per resident from their own history; the model does not use other residents or facility availability.
- Cold-start residents (no history) get global patterns only; there are none in this test split, so that case is untested here.
- The 30-minute nudge window and exact-hour rule are choices made for this write-up; a different tolerance changes the headline numbers.

6. Reproducing the results

See `README.md`. In short: `pip install -r requirements.txt`, then `PYTHONPATH=src python -m facility_pred.run all` (optionally regenerating the data first with `python -m facility_pred.data_gen`). The three steps are also available separately: `run train`, `run predict` (predictions only, `outputs/predictions.csv`) and `run evaluate`.